

Лабораторная работа № 5

Тема: Создание и использование middleware в Express.js

Цель: Научиться создавать собственные middleware-функции, использовать req, res и next, управлять областью действия middleware, изменять объект запроса и останавливать или продолжать обработку HTTP-запроса.

Практическое задание

Продолжить проект **Internet Shop API**.

Необходимо добавить в приложение несколько middleware:

- глобальное логирование запросов;
- добавление времени запроса;
- обработку JSON;
- проверку условного API-ключа для создания товаров;
- Router-level middleware для ресурса products.

Последовательность выполнения

1. Создайте каталог middleware.

Итоговая структура:

```
express-shop/  
├── middleware/  
│   ├── logger.js  
│   ├── requestTime.js  
│   └── checkApiKey.js  
├── routes/  
│   ├── productRoutes.js  
│   ├── categoryRoutes.js  
│   └── orderRoutes.js  
├── app.js  
├── package.json  
└── package-lock.json
```

2. Создайте middleware logger.

Файл:

middleware/logger.js

Реализуйте функцию:

```
export default function logger(req, res, next) {  
  console.log(  
    new Date().toISOString(),  
    req.method,  
    req.originalUrl  
  );  
  
  next();  
}
```

Middleware должно выводить в терминал информацию о каждом запросе.

3. Подключите logger глобально.

В app.js:

```
import logger from "../middleware/logger.js";
```

```
app.use(logger);
```

Запустите сервер и выполните несколько запросов:

```
GET /api/products
```

```
GET /api/categories
```

```
GET /api/orders
```

Убедитесь, что каждый запрос появляется в терминале.

4. Создайте middleware requestTime.

Файл:

```
middleware/requestTime.js
```

Код:

```
export default function requestTime(req, res, next) {  
  req.requestTime = new Date().toISOString();  
  next();  
}
```

Подключите его глобально:

```
app.use(requestTime);
```

5. Используйте req.requestTime в маршруте.

Добавьте:

```
GET /api/status
```

Ответ должен иметь вид:

```
{  
  "status": "ok",  
  "requestTime": "2026-09-16T10:30:00.000Z"  
}
```

Конкретное время должно формироваться автоматически.

6. Проверьте порядок middleware.

В app.js расположите подключения примерно так:

```
app.use(logger);
```

```
app.use(requestTime);
```

```
app.use(express.json());
```

```
app.use("/api/products", productRoutes);
```

```
app.use("/api/categories", categoryRoutes);
```

```
app.use("/api/orders", orderRoutes);
```

Добавьте временные `console.log()` в несколько middleware и определите фактический порядок их выполнения.

7. Создайте checkApiKey.js.

Реализуйте учебную проверку заголовка:

```
export default function checkApiKey(req, res, next) {  
  const apiKey = req.get("x-api-key");  
  
  if (apiKey !== "college-demo") {  
    return res.status(403).json({  
      message: "Access denied"  
    });  
  }  
  
  next();  
}
```

Это только учебная демонстрация middleware, а не полноценная система аутентификации.

8. Защитите только создание товара.

В productRoutes.js импортируйте middleware:

```
import checkApiKey from "../middleware/checkApiKey.js";
```

Подключите его только к:

POST /api/products

Пример структуры:

```
router.post("/", checkApiKey, (req, res) => {  
  const product = {  
    id: products.length + 1,  
    ...req.body  
  };  
  
  products.push(product);  
  
  res.status(201).json(product);  
});
```

9. Проверьте запрос без API-ключа.

Отправьте:

POST /api/products

с JSON-телом, но без x-api-key.

Ожидаемый статус:

403 Forbidden

Ожидаемый ответ:

```
{  
  "message": "Access denied"  
}
```

10. Проверьте неправильный API-ключ.

Добавьте заголовок:

x-api-key: wrong-key

Результат также должен быть:

403 Forbidden

Обработчик создания товара выполняться не должен.

11. Проверьте правильный API-ключ.

Передайте:

x-api-key: college-demo

и JSON:

```
{  
  "name": "SSD 1TB",  
  "price": 45000,  
  "category": "Components",  
  "inStock": true  
}
```

Middleware должно вызвать next(), после чего товар создаётся.

Ожидаемый статус:

201 Created

12. Создайте Router-level middleware для products.

В productRoutes.js перед маршрутами добавьте:

```
router.use((req, res, next) => {  
  console.log("Product Router:", req.method, req.originalUrl);  
  next();  
});
```

Проверьте:

GET /api/products

GET /api/products/1

POST /api/products

Сообщение должно появляться для routes товаров.

13. Убедитесь, что Product Router middleware не работает для других ресурсов.

Выполните:

GET /api/categories

GET /api/orders

Сообщение:

Product Router:

для них появляться не должно.

14. Проведите эксперимент без next().

Временно удалите:

next();

из Router-level middleware.

Выполните:

GET /api/products

Посмотрите поведение HTTP-клиента.

После эксперимента обязательно верните next().

Объясните, почему конечный обработчик не был выполнен.

15. Проведите эксперимент с порядком `express.json()`.

Правильный вариант:

```
app.use(express.json());
```

```
app.use("/api/products", productRoutes);
```

Затем временно поменяйте порядок:

```
app.use("/api/products", productRoutes);
```

```
app.use(express.json());
```

Отправьте POST-запрос с JSON и сравните поведение `req.body`.

После эксперимента восстановите правильный порядок.

Требования к результату

Приложение должно:

- содержать отдельный каталог `middleware`;
- иметь собственный глобальный `logger`;
- добавлять `requestTime` в объект `req`;
- использовать `express.json()` до маршрутов;
- иметь `middleware checkApiKey`;
- возвращать 403, если проверка не пройдена;
- вызывать `next()`, если запрос разрешён;
- использовать `route-level middleware` для `POST /api/products`;
- использовать `Router-level middleware` внутри `productRoutes`;
- сохранять работоспособность существующих `routes`.